# 微信步数

## Part 1

最直接的思路是从每个位置出发逐步模拟，直到走出边界。将 $n$ 步移动称作一轮，如果从某个位置出发一轮后回到了原地，那么就永远都走不出来，这可以对于每个维度计算出一轮的总位移来判断。使用 $C, W$ 表示所有维度大小的乘积与最大值，时间复杂度为 $O(CWn)$。

```
#include <bits/stdc++.h>
using namespace std;

const int N = 5e5 + 5, K = 12, P = 1e9 + 7;
int n, k, w[K], c[N], d[N];
int s[K], p[K], q[K] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1};

bool nxt() { //枚举下一个起点坐标
    int i = 1;
    while(q[i] == w[i]) q[i++] = 1;
    q[i]++;
    return i <= k;
}

int main() {
    int ans = 0, flag = 0; //标记能否走出
    scanf("%d%d", &n, &k);
    for(int j = 1; j <= k; j++) scanf("%d", &w[j]);

    for(int i = 1; i <= n; i++) {
        scanf("%d%d", &c[i], &d[i]);
        s[c[i]] += d[i]; //计算某个维度的位移
    }
    for(int j = 1; j <= k; j++) if(s[j] != 0) flag = 1;

    do {
        memcpy(p, q, sizeof(p)); //复制起点坐标

        for(int i = 1;; i = (i < n ? i + 1 : 1)) { //重复每一轮
            ans++, p[c[i]] += d[i];
            if(p[c[i]] < 1 || p[c[i]] > w[c[i]]) break; //走出边界

            if(i == n - 1 && !flag) { //一轮后回到原点
                puts("-1");
                return 0;
            }
        }
    } while(nxt());

    printf("%lld\n", ans);
    return 0;
}
```

## Part 2

考虑从所有的位置一起出发，每走一步就会有一些位置走出边界，而在每个维度上剩余位置必然是一段连续的区间，这些区间长度的乘积就是能走下一步的数量。维护每个维度上当前两个方向的最大位移即可推算区间长度，当在某个维度上全部走出时即可结束，时间复杂度为 $O(Wn)$。

```cpp
#include <bits/stdc++.h>
using namespace std;

const int N = 5e5 + 5, K = 12, P = 1e9 + 7;
int n, k, w[K], c[N], d[N];
int s[K], l[K], r[K];

int main() {
    long long ans = 1, flag = 0; //标记能否走出

    scanf("%d%d", &n, &k);
    for(int j = 1; j <= k; j++) {
        scanf("%d", &w[j]);
        ans = ans * w[j] % P; //统计走出第一步的数量
    }

    for(int i = 1; i <= n; i++) {
        scanf("%d%d", &c[i], &d[i]);
        s[c[i]] += d[i]; //计算某个维度的位移
    }
    for(int i = 1; i <= k; i++) if(s[i] != 0) flag = 1;

    memset(s, 0, sizeof(s));
    for(int i = 1;; i = (i < n ? i + 1 : 1)) { //重复每一轮
        int j = c[i];
        s[j] += d[i]; //累计位移
        l[j] = min(l[j], s[j]), r[j] = max(r[j], s[j]);

        long long cnt = 1; //还在场地内的数量
        for(int j = 1; j <= k; j++) {
            if(w[j] + l[j] - r[j] <= 0) { //全部走出边界
                printf("%lld\n", ans);
                return 0;
            }
            cnt = cnt * (w[j] + l[j] - r[j]) % P;
        }

        ans = (ans + cnt) % P;
        if(i == n - 1 && !flag) { //一轮后回到原点
            puts("-1");
            return 0;
        }
    }
}
```

## Part 3

考虑能否快速计算出移动的轮数，在一个维度上，除第一轮外，之后每过一轮走出边界的数量就是这个维度一轮的位移。使用 $w[j][i]$ 表示移动 $i$ 步后第 $j$ 个维度留下来的区间长度，那么当 $i > n$ 时,$w[j][i] - w[j][i+n]$ 是一个定值，即构成一个等差数列。于是，只需模拟第一轮即可。